Data processing

Data processing for Juliet Test Suite for C/C++

In this project, raw data were obtained from the Juliet Test Suite for C/C++ vulnerabilities. This dataset contains large-scale source code files organized according to the Common Weakness Enumeration (CWE) type, and vulnerable and safe codes are separated. However, since the original files were unstructured and not suitable for direct use in machine learning, a systematic data processing pipeline was established and preprocessed. The main steps and reasons are as follows.

1. **File Collection and Initial Cleaning**
   All source code files with .c, .cpp, and .h extensions were recursively collected from the testcases folder using Python's os library. In this process, non-related files such as README, Makefile, and text documents were excluded to leave only code samples. Through this, unnecessary noise in the dataset was reduced and the quality was improved.
2. **Removing Empty and Duplicate Files**
   The codes collected from the files and the path of the file were converted into a DataFrame structure using Pandas. Through this process, each file path and code content were systematically stored, and data processing could be easily applied in the next steps. Then, the empty file was deleted, and the duplicate samples showing the same code in other file were deleted through the drop_duplicates function. By going through this refining process, the dataset was composed of only unique and valid codes, and bias that may occur due to duplicate data in the learning stage could be prevented.
3. **Code-Level Cleaning**

   In code-level data cleaning, comments in the code were first removed. All block comments and line comments were deleted using regular expressions. Through this, feature engineering was not affected by unnecessary data. Subsequently, the whitespace was normalized. Several types of spaces (ex. double spaces, tabs) were unified so that the entire code remained in a constant format. Through this data processing, the dataset was able to have a clean and consistent form and reduce noise.
4. **Function-Level Extraction**
   In many cases, multiple functions were included in a single source file. To solve this problem, the code was separated into individual function units using a regular expression.
   In this way, each data row could have only one function, and the model could learn vulnerabilities at the function level rather than the entire file.

5. **Metadata Creation**

   After code refinement and function unit separation, additional metadata was generated for each function unit sample. First, each row was given a unique ID so that individual samples could be identified within the dataset. Second, programming language information was added. Based on the file extension, .c and .h files were classified as C language, and .cpp files were classified as C++ language. For other cases, the dataset was maintained consistency by processing it as "Unknown". Third, the vulnerability type was extracted. The file path of the Juliet Test Suite includes a vulnerability type along with a Common Weakness Enumeration (CWE) number, and the vulnerability type string following the CWE number was extracted using a regular expression. For example, only the Stack_Based_Buffer_Overflow part was extracted from the path name CWE121_Stack_Buffer_Overflow and stored in a new column. If the information did not exist, it was replaced with "Unknown".

6. **Labeling**

   The name of the functions included good or bad to distinguish between the vulnerable code and the safe code. Each function data was labeled using the characteristics of these data. This automatic labeling method made it possible to construct data in the form of supervised learning to be used by the ML model later by utilizing the Juliet database structure.

7. **Handling Missing Values**

   Rows with the label 'unknown' were removed first. This is because if incorrect labels or incomplete codes are included, noise may be caused during the model training process and vulnerability detection accuracy may be degraded.
   However, when metadata such as language information was missing, a default value such as "Unknown" was assigned. This makes the dataset complete without losing essential columns.
   There are several techniques for processing missing values, such as mean substitution and regression-based prediction, but in this project, it was more reasonable to remove incomplete or unreliable samples because accurate labels and functional code blocks are key elements due to the nature of code vulnerability detection. This approach has contributed to ensuring data quality and minimizing unnecessary biases that may occur in the learning process.

8. **Encoding and Transformation**

   The vulnerable label data was converted into an integer and mapped for safety to 0 and for vulnerable to 1.
   In addition, programming language metadata was converted into binary columns by applying one-hot encoding. Through this transformation, categorical data was reorganized into a form that can be directly used in machine learning algorithms using numeric data.

9. **Tokenization and Normalization**
   Each function was decomposed into identifier, keyword, operator, and literal units. After that, the number was normalized to <NUM>, the string to <STR>, and the identifier excluding the reserved word(keywords) was normalized to <VAR>.
   It is difficult for the model to understand directly because the original source code is a string, but if separated and normalized into such meaningful units, noise is reduced and the model can recognize the structure and pattern of the code.
10. **Quality Filtering**
    Very short or trivial functions with fewer than 10 tokens were removed. This prevented meaningless samples from degrading model accuracy.

Data processing for basic_data_3.jsonl

The basic_data_3.jsonl file is composed of JSON Lines format, so each line was first parsed and converted into DataFrame. This process is suitable for efficiently fetching large-scale code samples. After that, only the key column (language, flexibility_type, code_snippet) required for analysis was left, and the body of the code was changed to a standardized name called code. Each row was given a unique ID so that samples can be tracked within the dataset, and the label was designated as 1 collectively because all of the characteristics of the basic dataset are provided as vulnerable codes.
In the next step, the code string was decomposed into token units, and numbers, strings, and variable names were normalized into generalized tokens such as <NUM>, <STR>, and <VAR>. This is to convert the raw code into a pattern that can be learned by dividing it into semantic units because it is difficult for the model to understand the structure with a simple string. Subsequently, short code pieces with too few tokens were removed. This is a quality management process to prevent meaningless samples from being included in the model training and thus hindering performance. Finally, the programming language column was converted into binary features such as lang_C and lang_C++ by applying one-hot encoding. In this way, categorical data is converted into numerical types and can be used directly in machine learning algorithms.
Through this series of preprocessing processes, the basic dataset has been refined so that the function unit code has a consistent format and structure. It also laid the foundation for the model to effectively learn the patterns of vulnerable codes through tokenization, normalization, and language encoding.

11. **Feature Engineering with TF-IDF**

    In this project, characteristic engineering using TF-IDF vectorization was performed to classify vulnerability by function unit of source code. First, the source code was divided into function units, and then tokenization work was performed for each function. This token list was converted into a string to convert it into a form suitable for TF-IDF vectorization.
    By integrating the token strings of the Juliet dataset and the Basic dataset into a single corpus, IDF values were learned based on the entire dataset. This allowed us to build a vectorization model that can reflect both common and rare tokens between the two datasets.
    In this process, we used scikit-learn's TfidVectorizer, and set min_df=5 to reduce noise and highlight common patterns by removing rare tokens that appear only in fewer than five functions.
    TF-IDF quantifies the importance of each token by multiplying how often it appears within a particular function (TF) and how rare of all functions (IDF). Common tokens, for example, such as if, int, and return, are given a low weight because they appear in most functions. On the other hand, rare tokens that appear only in certain vulnerable codes are highly weighted, helping classification models learn important patterns better.
    The vectorized results are stored in the form of a sparse matrix, where each row represents one function, each column represents one token, and the value represents the TF-IDF weight of that token. This matrix is transformed into DataFrame and then merged with the original metadata to form a form that can be used directly in the machine learning model.
    Finally, we merge the basic dataset with TF-IDF vectorization into a single DataFrame, remove the existing ID column, and redefine the new sequential ID. This integrated dataset simultaneously reflects the frequency and rarity of the code to provide the precise characteristics needed for vulnerability classification and has the right input form for numerical vector-based machine learning models.

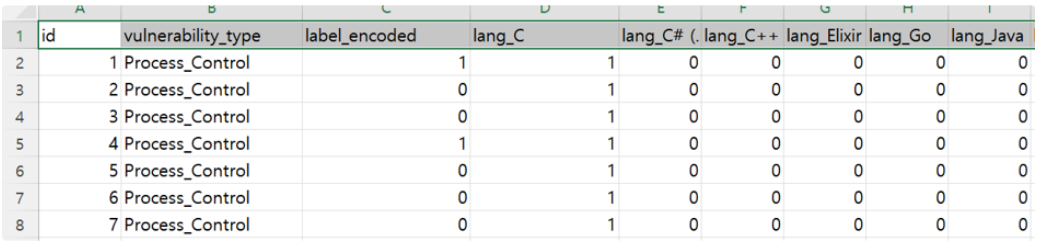
12. **Column reordering**

    Juliet and Basic datasets with TF-IDF vectorization are merged into one DataFrame, the existing ID columns are removed and new sequential IDs are redefined. The main columns (id, flexibility_type, label_encoded, lang_related columns, etc.) required for model training are finally aligned forward, and the remaining TF-IDF characteristic columns are placed at the back for better readability and processing efficiency.

11. **Final Dataset Export**

    The processed dataset was exported into both CSV and JSONL formats. These formats ensured compatibility with a wide range of machine learning tools and simplified loading for training and evaluation.

Column Descriptions (processed_dataset_final.csv)



	A	B	C	D	E	F	G	H	I
1	id	vulnerability_type	label_encoded	lang_C	lang_C# (.	lang_C++	lang_Elixir	lang_Go	lang_Java
2	1	Process_Control	1	1	0	0	0	0	0
3	2	Process_Control	0	1	0	0	0	0	0
4	3	Process_Control	0	1	0	0	0	0	0
5	4	Process_Control	1	1	0	0	0	0	0
6	5	Process_Control	0	1	0	0	0	0	0
7	6	Process_Control	0	1	0	0	0	0	0
8	7	Process_Control	0	1	0	0	0	0	0

Q	R	S	T	U	V	W	X	Y	Z	AA	AB	A
lang_Scala	break	case	catch	char	class	const	default	delete	do	double	else	FAL
0	0.122223365954304	0	0	0.1125242	0	0	0	0	0.0345445	0	0.0234615	
0	0	0	0	0.1344901	0	0	0	0	0	0	0.084124	
0	0	0	0	0	0	0	0	0	0	0	0	
0	0.121646496036862	0	0	0.1119931	0	0	0	0	0.034382	0	0.023351	
0	0	0	0	0.1182463	0	0	0	0	0	0	0.1479276	
0	0	0	0	0.1334226	0	0	0	0	0	0	0.083457	
0	0	0	0	0	0	0	0	0	0	0	0	
0	0.121350045466226	0	0	0.1117202	0	0	0	0	0.034298	0	0.0232938	

1. **id**
   - Unique identifier assigned to each code sample (function-level).
2. **vulnerability_type**
   - Indicating the type of code vulnerability.
3. **label_encoded**
   - Encoded label representing whether the code is vulnerable.
   - `0` = safe code
   - `1` = vulnerable code
4. **lang_###**
   - One-hot encoded indicator for the programming language.
   - `1` if the sample is written in ###, otherwise `0`.
5. **TF-IDF Token Columns**

Each of the following columns stores the **TF-IDF score** of the corresponding keyword.

- `0.0` → keyword does not appear in the function.
- Higher values → keyword is frequent in this function but rare across the dataset (more distinctive).

Summary

Through this multi level pipeline, we created the final dataset for use in ML models. This dataset has been processed into a clean and consistent form that can represent vulnerable and safe codes. Each pre-processing step is designed with the aim of improving the quality of the

training data and reducing noise and duplication. Through data cleaning transformation, labeling, token normalization, and TF-IDF-based feature engineering, raw data was changed to dataset to be used in ML models.